

MLOPS IN PRODUCTION · ISSUE 07 OF 08

Batch vs real-time serving.

Choose batch unless the decision happens inside the request.

07

AR

Akhilesh Ranjan Singh

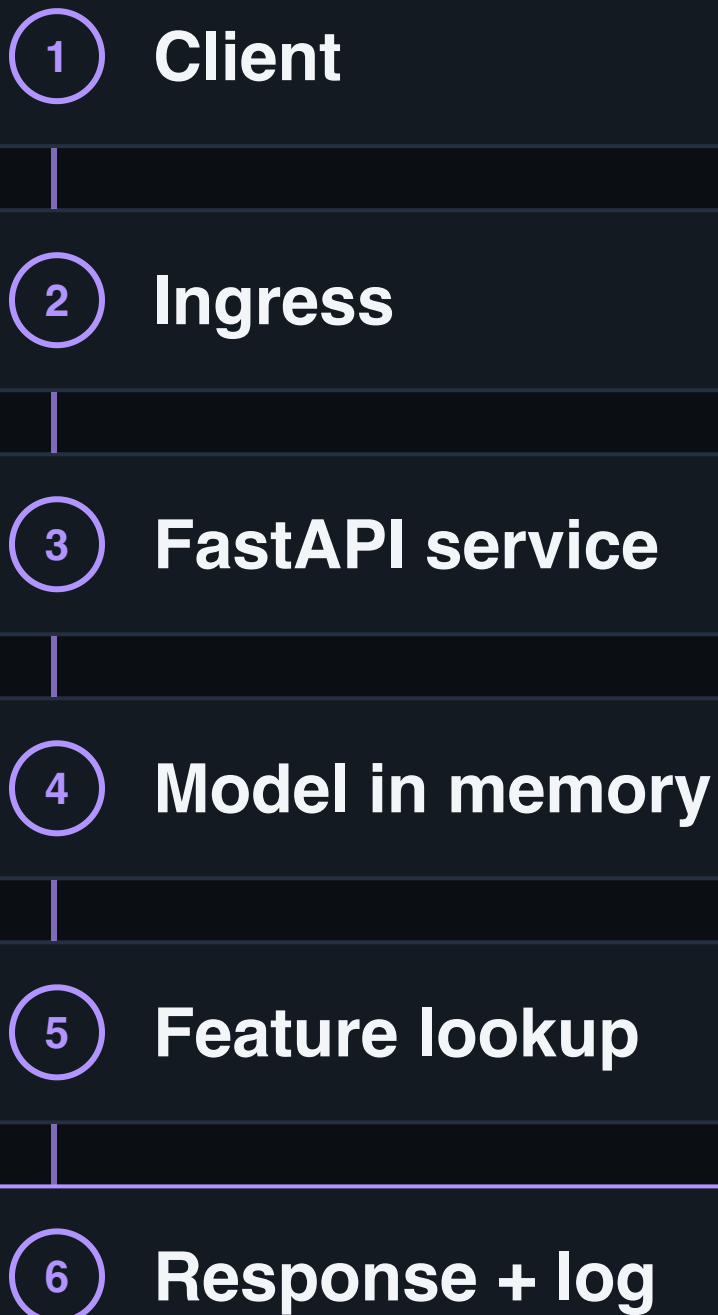
Senior DevSecOps Engineer · github.com/iarsingh

Swipe →

Batch or real-time?

	BATCH	REAL-TIME
Latency	Minutes to hours	Milliseconds
Cost	Bursty compute	Always on
Features	Precomputed	Online store
Failure	Rerun the job	Needs a fallback
Fits	Scoring lists	In-request decisions

Real-time path



Log every prediction with its model version for monitoring and audit.

Load once, report readiness

PYTHON

```
from contextlib import asynccontextmanager
from fastapi import FastAPI, HTTPException

state = {}

@asynccontextmanager
async def lifespan(app):
    state["model"] = load_model("models:/churn@champion")
    yield

app = FastAPI(lifespan=lifespan)

@app.get("/readyz")
def ready():
    if "model" not in state:
        raise HTTPException(503)
    return {"ok": True}
```

When latency climbs

01 Model loaded per request

Load at startup, keep it in memory.

02 CPU throttling

Check CFS throttling against limits.

03 Synchronous feature calls

Batch or cache lookups.

04 Cold starts on scale-up

Warm replicas and startup probes.



Serving baseline

- ☒ Readiness waits for model load
- ☒ Model version in every log line
- ☒ Timeout and fallback defined
- ☒ Requests sized from load tests
- ☒ p95 latency SLO with alert



KEY TAKEAWAY

Batch by default. Real-time when the request needs the answer.

- Compare latency, cost and failure modes
- Load the model once, gate readiness
- Throttling and cold starts cause most latency

YOUR TURN

Which of your real-time models could have been a batch job?

FOLLOW AKHILESH RANJAN SINGH

Next in the series: #08 Vertex AI vs self-managed GKE

[Save](#) · [Repost](#)